Intelligent Control

Session 2: Linear Systems Review

Dr. Gerardo Flores

SENG 5360 · Texas A&M International University ·

Today

1. Solving $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$: the matrix exponential
2. Eigenvalues and stability
3. State feedback $\boldsymbol{u} = -\boldsymbol{K}\boldsymbol{x}$
4. Simulation lab

Learning objectives

- write the solution of $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$ in closed form and explain what $e^{\boldsymbol{A}t}$ means;
- assess stability from the eigenvalues and, when necessary, the associated Jordan structure;
- relate a complex eigenvalue pair to a decay rate and an oscillation frequency;
- design a gain $\boldsymbol{K}$ to assign the closed-loop poles when $(\boldsymbol{A}, \boldsymbol{B})$ is controllable.

Where we left off

Last session we built the template

$$\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{u}$$
$$\boldsymbol{y} = \boldsymbol{C}\boldsymbol{x} + \boldsymbol{D}\boldsymbol{u}$$

and claimed two things about it without proof:

- it has a closed-form solution;
- the origin is exponentially stable iff all eigenvalues have negative real part. For *Lyapunov* stability with eigenvalues on the imaginary axis, the Jordan blocks must also be checked.

Read the second claim carefully. It concerns the unforced system $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$, that is $\boldsymbol{u} = \boldsymbol{0}$. With an input the state also depends on $\boldsymbol{u}(t)$, and if $\boldsymbol{u}$ is a feedback of the state, as in $\boldsymbol{u} = -\boldsymbol{K}\boldsymbol{x}$, the relevant eigenvalues become those of $\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K}$. Today we earn both claims.

Keep this example in view

The mass-spring-damper of Session 1, with $m = 1$, $c = 3$, $k = 2$:

$$\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}, \quad \boldsymbol{B} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

We solve it, analyse it, and then control it, all in this session.

1. Solving the linear equation

Start with one dimension

Take a scalar state $x(t) \in \mathbb{R}$ and a scalar constant $a \in \mathbb{R}$ (one state, one number, no matrices yet):

$$\dot{x} = ax, \qquad x(0) \text{ given}$$

Where the solution comes from

1. Assume $x(0) \neq 0$ and separate: $\dfrac{dx}{x} = a\,dt$. (If $x(0) = 0$, $x(t) \equiv 0$.)

2. Integrate from 0 to t: $\ln|x(t)| - \ln|x(0)| = at$.

3. Exponentiate: $|x(t)/x(0)| = e^{at}$. The sign cannot change: if $x(t^\star) = 0$ at some $t^\star$, uniqueness of solutions would force $x(t) \equiv 0$, contradicting $x(0) \neq 0$.

4. Therefore,

$$\boxed{x(t) = e^{at}x(0)}$$

Check: $\dot{x} = ae^{at}x(0) = a\,x(t)$. ✓

Two readings, both of which survive into the matrix case:

- e^{at} maps the initial state to the state at time t;
- the sign of a decides everything: $a < 0$ decays, $a > 0$ blows up, $a = 0$ stays put.

Step 1 is what fails for a vector state: you cannot divide by a vector. That is why the **matrix case needs a different route**.

The matrix exponential

Definition

$$e^{\boldsymbol{A}t} \;=\; \boldsymbol{I} + \boldsymbol{A}t + \frac{(\boldsymbol{A}t)^2}{2!} + \frac{(\boldsymbol{A}t)^3}{3!} + \cdots \;=\; \sum_{k=0}^{\infty} \frac{(\boldsymbol{A}t)^k}{k!}$$

$k = 0, 1, 2, \ldots$ is the summation index, the sum runs to infinity, $(\boldsymbol{A}t)^k$ is $\boldsymbol{A}t$ multiplied by itself k times, $(\boldsymbol{A}t)^0 = \boldsymbol{I}$ and $0! = 1$.

The series converges for every square $\boldsymbol{A}$ and every t, and the result is a **matrix** the same size as $\boldsymbol{A}$. Properties:

- $e^{\boldsymbol{A}\cdot 0} = \boldsymbol{I}$;
- $\dfrac{d}{dt} e^{\boldsymbol{A}t} = \boldsymbol{A} e^{\boldsymbol{A}t}$, which is why it solves our equation;
- $\left(e^{\boldsymbol{A}t}\right)^{-1} = e^{-\boldsymbol{A}t}$;
- in general $e^{(\boldsymbol{A}+\boldsymbol{F})t}$ cannot be factored as $e^{\boldsymbol{A}t} e^{\boldsymbol{F}t}$; guaranteed only when $\boldsymbol{A}\boldsymbol{F} = \boldsymbol{F}\boldsymbol{A}$.

The trap: $e^{\boldsymbol{A}t}$ is not the entrywise exponential

For $\boldsymbol{A} = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$: ✗ wrong, entry by entry, $\left[\begin{smallmatrix} e^0 & e^t \\ e^0 & e^0 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 1 & e^t \\ 1 & 1 \end{smallmatrix}\right]$; ✓ right, from the series ($\boldsymbol{A}^2 = \boldsymbol{0}$), $e^{\boldsymbol{A}t} = \boldsymbol{I} + \boldsymbol{A}t = \left[\begin{smallmatrix} 1 & t \\ 0 & 1 \end{smallmatrix}\right]$. At $t = 0$ the wrong one gives $\left[\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right]$, not $\boldsymbol{I}$. In NumPy `np.exp(A*t)` is wrong, `scipy.linalg.expm(A*t)` is right.

The solution when A is a matrix

Now the state is a vector, $\boldsymbol{x}(t) \in \mathbb{R}^n$, and $\boldsymbol{A} \in \mathbb{R}^{n \times n}$.

Solution of $\dot{x} = Ax$, $x(0)$ given

$$\boldsymbol{x}(t) \;=\; e^{\boldsymbol{A}t}\,\boldsymbol{x}(0)$$

Why it is the solution

1. Initial condition: $\boldsymbol{x}(0) = e^{\boldsymbol{A}\cdot 0}\boldsymbol{x}(0) = \boldsymbol{I}\boldsymbol{x}(0)$. ✓
2. Differentiate, using the second property: $\frac{d}{dt}\left(e^{\boldsymbol{A}t}\boldsymbol{x}(0)\right) = \boldsymbol{A}e^{\boldsymbol{A}t}\boldsymbol{x}(0) = \boldsymbol{A}\boldsymbol{x}(t)$. ✓
3. It is the *only* solution, by uniqueness for linear ODEs.

The formula is identical to the scalar one, with e^{at} replaced by $e^{\boldsymbol{A}t}$.

The object $e^{\boldsymbol{A}t}$ is called the **state transition matrix**, written $\boldsymbol{\Phi}(t)$: hand it a state at time 0, it returns the state at time t. It is the standard name in the control literature, and the one you will meet again in observers and in the Kalman filter.

Board: differentiate the series term by term to check $\frac{d}{dt}e^{\boldsymbol{A}t} = \boldsymbol{A}e^{\boldsymbol{A}t}$. *Åström & Murray, 2008, §5.3.*

Reminder: eigenvalues and eigenvectors

Definition

$$\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}, \quad \boldsymbol{v} \neq \boldsymbol{0}$$ Applying A to v keeps it on the same line and scales it by λ.

Rewriting as $(\lambda \boldsymbol{I} - \boldsymbol{A})\boldsymbol{v} = \boldsymbol{0}$: a non-zero $\boldsymbol{v}$ exists only if that matrix is singular, so the eigenvalues are the roots of $\det(\lambda \boldsymbol{I} - \boldsymbol{A}) = 0$.

Diagonal is easy

$\boldsymbol{A} = \begin{bmatrix} 2 & 0 \\ 0 & -3 \end{bmatrix}$

$\lambda = 2, -3$ with $\boldsymbol{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\boldsymbol{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.

The axes are the invariant directions.

Our example

$\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$

$\lambda^2 + 3\lambda + 2 = 0 \Rightarrow \lambda = -1, -2.$

$\boldsymbol{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$, $\boldsymbol{v}_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$.

Check: $\boldsymbol{A}\boldsymbol{v}_1 = \begin{bmatrix} -1 \\ 1 \end{bmatrix} = -\boldsymbol{v}_1$. ✓

Complex ones exist

$\boldsymbol{A} = \begin{bmatrix} 0 & 2 \\ -2 & 0 \end{bmatrix}$

$\lambda^2 + 4 = 0 \Rightarrow \lambda = \pm 2j.$

No real direction survives: this matrix rotates.

A method for computing e^{At}: diagonalization

We have a formula, $\boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{x}(0)$, but not yet a *number*. What follows is a **method**: a procedure that takes $\boldsymbol{A}$ and returns a closed-form $e^{\boldsymbol{A}t}$ valid for all t, with the λ_i explicit in the exponents. It requires $\boldsymbol{A}$ to be diagonalizable, i.e. to have n independent eigenvectors.

1. Diagonalize. Put the n independent eigenvectors in the columns of $\boldsymbol{V}$ and the eigenvalues in $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$. Then $\boldsymbol{AV} = \boldsymbol{V\Lambda}$, that is

$$\boldsymbol{A} = \boldsymbol{V\Lambda V}^{-1}$$

2. Powers collapse, since the inner factors cancel:

$$\boldsymbol{A}^2 = \boldsymbol{V\Lambda}\underbrace{\boldsymbol{V}^{-1}\boldsymbol{V}}_{I}\boldsymbol{\Lambda V}^{-1} = \boldsymbol{V\Lambda}^2\boldsymbol{V}^{-1}$$

and repeating, $\boldsymbol{A}^k = \boldsymbol{V\Lambda}^k\boldsymbol{V}^{-1}$ for every $k = 0, 1, 2, \ldots$ of the series.

3. Substitute into the series and pull the constant matrices out of the sum:

$$e^{\boldsymbol{A}t} = \sum_{k=0}^{\infty} \frac{\boldsymbol{A}^k t^k}{k!} = \sum_{k=0}^{\infty} \frac{\boldsymbol{V\Lambda}^k\boldsymbol{V}^{-1}t^k}{k!}$$
$$= \boldsymbol{V}\left(\sum_{k=0}^{\infty} \frac{(\boldsymbol{\Lambda}t)^k}{k!}\right)\boldsymbol{V}^{-1} = \boldsymbol{V}\,e^{\boldsymbol{\Lambda}t}\,\boldsymbol{V}^{-1}$$

4. The diagonal case is easy. $\boldsymbol{\Lambda}^k$ is diagonal with entries λ_i^k, so entry i of the sum is the scalar series $\sum_k (\lambda_i t)^k/k! = e^{\lambda_i t}$:

$$e^{\boldsymbol{A}t} = \boldsymbol{V}\begin{bmatrix} e^{\lambda_1 t} & & \\ & \ddots & \\ & & e^{\lambda_n t} \end{bmatrix}\boldsymbol{V}^{-1}$$

The method in one line: eigenvalues and eigenvectors of $\boldsymbol{A}$ → build $\boldsymbol{V}$ → exponentiate the eigenvalues one by one → sandwich as $\boldsymbol{V}e^{\boldsymbol{\Lambda}t}\boldsymbol{V}^{-1}$. Step 4 is the *only* case where the entrywise exponential is correct: on a diagonal matrix.

The method applied, with numbers

Take $\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$.

1. $\det(\lambda \boldsymbol{I} - \boldsymbol{A}) = \lambda^2 + 3\lambda + 2 = 0$, so $\lambda_1 = -1$, $\lambda_2 = -2$.

2. Eigenvectors $\boldsymbol{v}_1 = [1,\ -1]^\top$, $\boldsymbol{v}_2 = [1,\ -2]^\top$, placed as columns:

$$\boldsymbol{V} = \begin{bmatrix} 1 & 1 \\ -1 & -2 \end{bmatrix}, \qquad \boldsymbol{V}^{-1} = \begin{bmatrix} 2 & 1 \\ -1 & -1 \end{bmatrix}$$

3. Assemble $\boldsymbol{V}\, e^{\boldsymbol{\Lambda} t}\, \boldsymbol{V}^{-1}$ with $e^{\boldsymbol{\Lambda} t} = \begin{bmatrix} e^{-t} & 0 \\ 0 & e^{-2t} \end{bmatrix}$.

Multiplying out:

$$e^{\boldsymbol{A}t} = \begin{bmatrix} 2e^{-t} - e^{-2t} & e^{-t} - e^{-2t} \\ -2e^{-t} + 2e^{-2t} & -e^{-t} + 2e^{-2t} \end{bmatrix}$$

Two checks worth doing every time

At $t = 0$: every exponential is 1, and the entries give $\begin{bmatrix} 2-1 & 1-1 \\ -2+2 & -1+2 \end{bmatrix} = \boldsymbol{I}$. ✓

As $t \to \infty$: both e^{-t} and e^{-2t} vanish, so $e^{\boldsymbol{A}t} \to \boldsymbol{0}$ and every solution decays. That is stability, read straight off the exponents.

Does the method work for every A?

No. It works exactly when $\boldsymbol{A}$ is **diagonalizable**, that is when it has n linearly independent eigenvectors, because only then does $\boldsymbol{V}^{-1}$ exist.

When it always works

- n distinct eigenvalues: the common case, so the method usually applies.
- $\boldsymbol{A}$ real symmetric, $\boldsymbol{A} = \boldsymbol{A}^\top$. Always diagonalizable, even with repeated eigenvalues.
- Complex eigenvalues are fine: $\boldsymbol{V}$, $\boldsymbol{\Lambda}$ come out complex but $\boldsymbol{V} e^{\boldsymbol{\Lambda} t} \boldsymbol{V}^{-1}$ is real again.
- Repeated eigenvalues are *not* automatically fatal: $2\boldsymbol{I}$ has two independent eigenvectors.

When it fails

A repeated eigenvalue with *too few* eigenvectors. Such an $\boldsymbol{A}$ is called **defective**.

$\boldsymbol{A} = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$: $\lambda = 0, 0$ but $\operatorname{rank}(\boldsymbol{A} - 0\boldsymbol{I}) = 1$, so there is only one linearly independent eigenvector; the eigenspace is the line spanned by $[1, 0]^\top$. $\boldsymbol{V}$ is not invertible.

What to do instead

By hand: the Jordan form. A block $\boldsymbol{J} = \lambda \boldsymbol{I} + \boldsymbol{N}$ with $\boldsymbol{N}$ nilpotent gives $e^{\boldsymbol{J}t} = e^{\lambda t}\left(\boldsymbol{I} + \boldsymbol{N}t + \frac{\boldsymbol{N}^2 t^2}{2} + \cdots\right)$, a finite sum. For $\boldsymbol{J} = \left[\begin{smallmatrix} -1 & 1 \\ 0 & -1 \end{smallmatrix}\right]$ this is $e^{-t}\left[\begin{smallmatrix} 1 & t \\ 0 & 1 \end{smallmatrix}\right]$: the terms $t^k e^{\lambda t}$ we met earlier.

Numerically: `scipy.linalg.expm` works for *every* square $\boldsymbol{A}$ and never forms eigenvectors.

Investigate using an AI assistant

Practical note: a *generic* arbitrarily small perturbation destroys the defective structure and separates the repeated eigenvalue, though not every one does: changing the 1 above to $1 + \epsilon$ leaves $\lambda = 0, 0$ and the matrix still defective. Defective matrices matter because they appear in *idealised* models such as the double integrator, which is the example above.

Modes, step by step

Assume throughout this slide that A is diagonalizable. Its eigenvectors are then independent and form a basis of $\mathbb{C}^n$ (complex, since a real matrix can have complex eigenvectors). We know $\boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{x}(0)$; the goal is to see what that expression contains.

Step 1. Write $\boldsymbol{x}(0)$ in that basis, as a combination of the eigenvectors:

$$\boldsymbol{x}(0) = c_1\boldsymbol{v}_1 + \cdots + c_n\boldsymbol{v}_n = \boldsymbol{V}\boldsymbol{c} \implies \boldsymbol{c} = \boldsymbol{V}^{-1}\boldsymbol{x}(0)$$

Step 2. What $e^{\boldsymbol{A}t}$ does to *one* eigenvector. From $\boldsymbol{A}\boldsymbol{v} = \lambda\boldsymbol{v}$ we get $\boldsymbol{A}^k\boldsymbol{v} = \lambda^k\boldsymbol{v}$. Feed that into the series:

$$e^{\boldsymbol{A}t}\boldsymbol{v} = \left(\boldsymbol{I} + \boldsymbol{A}t + \tfrac{\boldsymbol{A}^2t^2}{2!} + \cdots\right)\boldsymbol{v} = \boldsymbol{v} + \lambda t\,\boldsymbol{v} + \tfrac{\lambda^2t^2}{2!}\boldsymbol{v} + \cdots$$
$$= \underbrace{\left(1 + \lambda t + \tfrac{(\lambda t)^2}{2!} + \cdots\right)}_{e^{\lambda t}}\boldsymbol{v} = e^{\lambda t}\boldsymbol{v}$$

Step 3. Apply $e^{\boldsymbol{A}t}$ to the combination, distributing over the sum and the scalars:

$$\boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{x}(0) = e^{\boldsymbol{A}t}\left(\sum_i c_i\boldsymbol{v}_i\right) = \sum_i c_i\, e^{\boldsymbol{A}t}\boldsymbol{v}_i$$

Step 4. Use Step 2 on each term.

$$\boxed{\boldsymbol{x}(t) = \sum_{i=1}^{n} c_i e^{\lambda_i t}\boldsymbol{v}_i} \qquad \text{over } \mathbb{C}^n$$

Each term is a **modal contribution**. The real part of λ_i determines growth or decay, its imaginary part determines oscillation, and $\boldsymbol{v}_i$ determines the associated spatial direction. For real systems, conjugate terms combine into a real oscillatory mode.

Every step relies on linearity, which fails for a general nonlinear system. For a **defective** $\boldsymbol{A}$ linearity still holds, but the eigenvectors do not form a complete basis: generalized eigenvectors are required and the modal terms carry factors $t^k e^{\lambda t}$.

Modes on our example

$\boldsymbol{A} = \left[\begin{smallmatrix} 0 & 1 \\ -2 & -3 \end{smallmatrix}\right]$ with $\lambda_1 = -1$, $\boldsymbol{v}_1 = \left[\begin{smallmatrix} 1 \\ -1 \end{smallmatrix}\right]$ and $\lambda_2 = -2$, $\boldsymbol{v}_2 = \left[\begin{smallmatrix} 1 \\ -2 \end{smallmatrix}\right]$, so $\boldsymbol{V}^{-1} = \left[\begin{smallmatrix} 2 & 1 \\ -1 & -1 \end{smallmatrix}\right]$.

Start from $x(0) = [1, 0]^\top$

1. $\boldsymbol{c} = \boldsymbol{V}^{-1}\boldsymbol{x}(0) = \left[\begin{smallmatrix} 2 \\ -1 \end{smallmatrix}\right]$.
2. Assemble the two modes:

$$\boldsymbol{x}(t) = 2e^{-t}\begin{bmatrix} 1 \\ -1 \end{bmatrix} - 1e^{-2t}\begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

3. Componentwise: $x_1(t) = 2e^{-t} - e^{-2t}$, $x_2(t) = -2e^{-t} + 2e^{-2t}$.

Check: that is exactly the *first column* of the $e^{\boldsymbol{A}t}$ we computed two slides ago, which is what $e^{\boldsymbol{A}t}[1, 0]^\top$ must give. At $t = 0$ it returns $[1, 0]^\top$.

A mode that never appears

Now start from $\boldsymbol{x}(0) = \boldsymbol{v}_2 = [1, -2]^\top$. Then $\boldsymbol{c} = \boldsymbol{V}^{-1}\boldsymbol{x}(0) = \left[\begin{smallmatrix} 0 \\ 1 \end{smallmatrix}\right]$, so

$$\boldsymbol{x}(t) = e^{-2t}\begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

The slow mode has $c_1 = 0$: it is not excited and never shows up. This solution decays at e^{-2t}, twice as fast as a generic one.

If one excited mode has a strictly larger real part than the rest, that mode dominates; if several share the dominant real part, as a conjugate pair does, their combination does. Here -1 is strictly dominant. A generic $\boldsymbol{x}(0)$ excites both modes.

Polynomial factors from Jordan blocks

We want $e^{\boldsymbol{A}t}$ because $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}, \quad \boldsymbol{x}(0) = \boldsymbol{x}_0 \quad \Longrightarrow \quad \boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{x}_0$. If $\boldsymbol{A}$ is defective, we use

$$\boldsymbol{A} = \boldsymbol{V}\boldsymbol{J}\boldsymbol{V}^{-1}, \qquad e^{\boldsymbol{A}t} = \boldsymbol{V}e^{\boldsymbol{J}t}\boldsymbol{V}^{-1}.$$

Thus, we compute $e^{\boldsymbol{J}t}$ block by block.

Step 1. For a Jordan block of size r,

$$\boldsymbol{J}_\lambda = \lambda\boldsymbol{I} + \boldsymbol{N}, \qquad \boldsymbol{N}^r = 0.$$

Here r is the length of the Jordan chain, not necessarily the dimension of the full system.

$$\boldsymbol{N} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ \vdots & & & \ddots & 1 \\ 0 & 0 & \cdots & 0 & 0 \end{bmatrix}$$

Step 2. Since $(\lambda\boldsymbol{I})\boldsymbol{N} = \boldsymbol{N}(\lambda\boldsymbol{I})$,

$$e^{\boldsymbol{J}_\lambda t} = e^{(\lambda\boldsymbol{I}+\boldsymbol{N})t} = e^{\lambda t}e^{\boldsymbol{N}t}.$$

Step 3. Since $\boldsymbol{N}^k = 0$ for all $k \geq r$, the series stops:

$$e^{\boldsymbol{N}t} = \boldsymbol{I} + \boldsymbol{N}t + \frac{\boldsymbol{N}^2t^2}{2!} + \cdots + \frac{\boldsymbol{N}^{r-1}t^{r-1}}{(r-1)!}.$$

Therefore,

$$\boxed{e^{\boldsymbol{J}_\lambda t} = e^{\lambda t}\left(\boldsymbol{I} + \boldsymbol{N}t + \frac{\boldsymbol{N}^2t^2}{2!} + \cdots + \frac{\boldsymbol{N}^{r-1}t^{r-1}}{(r-1)!}\right)}$$

Key point: A Jordan block of size r can produce modal terms up to $t^{r-1}e^{\lambda t}$. These polynomial factors appear because the block has a nilpotent part $\boldsymbol{N}$, which comes from generalized eigenvectors.

Example: block size versus number of eigenvectors

Consider

$$\boldsymbol{A} = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}.$$

The only eigenvalue is

$$\lambda = 2,$$

with algebraic multiplicity 3.
Eigenvectors satisfy

$$(\boldsymbol{A} - 2\boldsymbol{I})\boldsymbol{v} = 0, \qquad \boldsymbol{A} - 2\boldsymbol{I} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Thus $v_2 = 0$, so there are two independent eigenvectors, for example

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Therefore, there are two Jordan blocks:

$$\boldsymbol{J}_1 = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}, \qquad \boldsymbol{J}_2 = [2].$$

Their sizes are

$$r_1 = 2, \qquad r_2 = 1.$$

The block with $r = 2$ produces

$$e^{2t}, \qquad te^{2t}.$$

The block with $r = 1$ produces only

$$e^{2t}.$$

Key point: the number of independent eigenvectors gives the number of Jordan blocks. The value r is the size of one block, and it determines the highest power of t that can appear.

Example: block size versus number of eigenvectors [continuation]

For the previous matrix,

$$\boldsymbol{A} = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix} = \boldsymbol{J}_1 \oplus \boldsymbol{J}_2, \qquad \boldsymbol{J}_1 = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}, \quad \boldsymbol{J}_2 = [2].$$

For the size-2 block,

$$\boldsymbol{J}_1 = 2\boldsymbol{I} + \boldsymbol{N}, \qquad \boldsymbol{N} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad \boldsymbol{N}^2 = 0.$$

Therefore,

$$e^{\boldsymbol{J}_1 t} = e^{2t}\,(\boldsymbol{I} + \boldsymbol{N}t) = e^{2t}\begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}.$$

For the size-1 block,

$$e^{\boldsymbol{J}_2 t} = e^{2t}.$$

Combining both blocks,

$$e^{\boldsymbol{A}t} = \begin{bmatrix} e^{\boldsymbol{J}_1 t} & 0 \\ 0 & e^{\boldsymbol{J}_2 t} \end{bmatrix} = e^{2t}\begin{bmatrix} 1 & t & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Finally,

$$\boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{x}(0) = e^{2t}\begin{bmatrix} 1 & t & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x_1(0) \\ x_2(0) \\ x_3(0) \end{bmatrix} = e^{2t}\begin{bmatrix} x_1(0) + t x_2(0) \\ x_2(0) \\ x_3(0) \end{bmatrix}.$$

2. Eigenvalues and stability

Stability of the origin

The origin is always an equilibrium of $\dot{x} = Ax$. Recall A is **Hurwitz** when every eigenvalue has strictly negative real part.

Stability of $\dot{x} = Ax$

Every eigenvalue satisfies $\mathrm{Re}(\lambda) < 0$	The origin is globally exponentially stable.
At least one eigenvalue satisfies $\mathrm{Re}(\lambda) > 0$	The origin is unstable.
All eigenvalues satisfy $\mathrm{Re}(\lambda) \leq 0$, and at least one has $\mathrm{Re}(\lambda) = 0$	The origin is not asymptotically stable. It is stable only if the modes on the imaginary axis remain bounded; otherwise, it is unstable.

Simulate/Investigate using an AI assistant

Hurwitz says more than "it goes to zero": the state is squeezed inside a decaying exponential envelope,

$$\|x(t)\| \leq Me^{-\alpha t}\|x(0)\|$$

where $\alpha > 0$ is the guaranteed decay rate, admissible for any $0 < \alpha < -\max_i \mathrm{Re}\,\lambda_i$ (equality too when the boundary eigenvalues are semisimple), and $M \geq 1$ is an overshoot allowance, since the state may grow before decaying.

Concrete M and α

For $A = \left[\begin{smallmatrix} 0 & 1 \\ -2 & -3 \end{smallmatrix}\right]$, $\alpha = 1$ *is* admissible because the dominant eigenvalue -1 is simple. In the Euclidean norm and its induced 2-norm the smallest constant is $M = \sqrt{10} \approx 3.16$.

Imaginary-axis eigenvalues: stable or unstable?

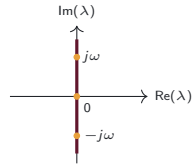
Suppose all eigenvalues satisfy

$$\mathrm{Re}(\lambda) \le 0,$$

and at least one lies on the imaginary axis.

$$\lambda = 0 \;\Rightarrow\; \text{a constant mode}, \qquad \lambda = \pm j\omega \;\Rightarrow\; \text{a sustained oscillation}.$$

These modes do not decay. Therefore, the origin is **not asymptotically stable**. We must still determine whether all trajectories remain bounded.



Eigenvalues on the imaginary axis

$A_1 = \left[\begin{smallmatrix} 0 & 0 \\ 0 & 0 \end{smallmatrix}\right]$: bounded

$$e^{A_1 t} = I, \qquad x(t) = x(0).$$

Every trajectory remains constant and bounded.

Stable, but not asymptotically stable.

$A_2 = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$: unbounded

$$e^{A_2 t} = \left[\begin{smallmatrix} 1 & t \\ 0 & 1 \end{smallmatrix}\right], \qquad x_1(t) = x_1(0) + t\,x_2(0).$$

For $x_2(0) \neq 0$, $x_1(t)$ grows linearly.

Unstable.

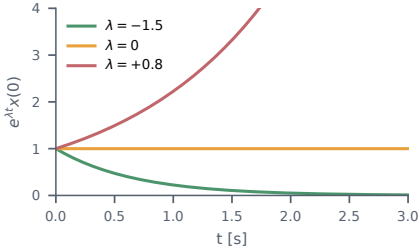
Key point: Both matrices have the same eigenvalues, $0, 0$, but only A_2 produces a growing factor t. Recall that a *mode* is a time-dependent component of the solution $x(t) = e^{At}x(0)$, such as $e^{\lambda t}$ or $te^{\lambda t}$. Modes associated with eigenvalues on the imaginary axis have no exponential decay. Therefore, the origin is stable only if all such modes remain bounded; if factors such as t or t^2 appear, the origin is unstable.

For a real eigenvalue, the sign decides

Each mode contributes $e^{\lambda t}$:

- $\lambda < 0$: decay, and the further left, the faster;
- $\lambda = 0$: neither decay nor growth;
- $\lambda > 0$: growth, the system runs away.

The origin is asymptotically stable only if *every* system mode decays; one eigenvalue in the right half plane loses stability however good the others are. A *particular* trajectory may still decay if its nondecaying modes are not excited.




```python
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 3, 400)

for lam in [-1.5, 0.0, 0.8]:   # <-- change these
    plt.plot(t, np.exp(lam*t), label=f"lam={lam}")

plt.ylim(0, 4); plt.xlabel("t [s]")
plt.legend(); plt.show()
```

Code 2.1

Time constant

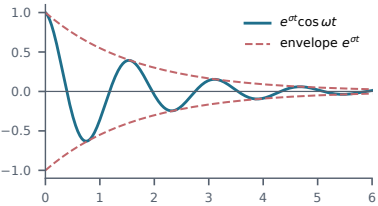
A mode with real $\lambda < 0$ falls to 37% in $\tau = 1/|\lambda|$ seconds and is essentially gone after 4τ. For $\lambda = -1.5$: $\tau = 0.67\,\text{s}$, settling in about 2.7 s. Try $\lambda = -0.2$ and count how long it takes.

Complex eigenvalues mean oscillation

Real matrices have complex eigenvalues in conjugate pairs, $\lambda = \sigma \pm j\omega_d$. The two modes combine into real solutions of the form $e^{\sigma t}\,(a\cos\omega_d t + b\sin\omega_d t)$:

Investigate using an AI assistant

- σ, the real part, sets the decay;
- ω_d, the imaginary part, is the damped oscillation frequency;
- stability depends on σ alone. Oscillation is not instability.



$\sigma = -0.6$, $\omega_d = 4$ rad/s

```python
from scipy.integrate import solve_ivp

sig, wd = -0.6, 4.0          # <-- change these
A = np.array([[ sig,  wd],
              [-wd,  sig]]) # eigenvalues sig +/- j*wd
print(np.linalg.eigvals(A))

sol = solve_ivp(lambda t, x: A @ x,
                [0, 6], [1., 0.], max_step=0.01)
plt.plot(sol.t, sol.y[0]); plt.show()
```

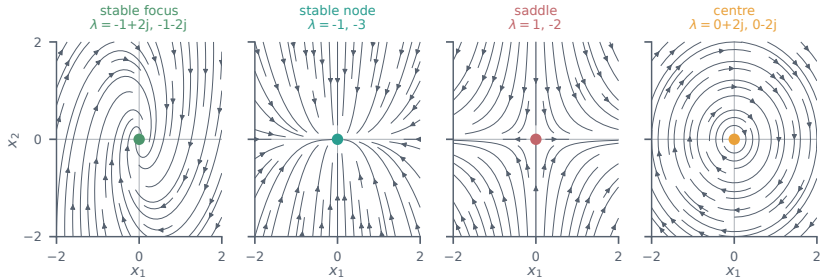
Code 2.2

Do not confuse ω_d with ω_n

The imaginary part is ω_d, *not* the natural frequency. They are related by $\omega_n = \sqrt{\sigma^2 + \omega_d^2} = |\lambda|$ and $\zeta = -\sigma/\omega_n$, which is the standard form $\lambda = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$. Here $\omega_d = 4$ but $\omega_n = \sqrt{0.36 + 16} = 4.04$. Try $\sigma = 0$ (never settles), then $\sigma = +0.3$ (grows while oscillating).

The four pictures worth memorising

These are **phase portraits**, also called phase-plane plots: the horizontal axis is the state x_1, the vertical axis is the state x_2, and time does not appear. Each curve is one trajectory, and the arrows show which way it is travelled.



- Focus: complex pair with $\sigma < 0$, spirals in. Node: two negative reals, no spiral.
- Saddle: one positive, one negative. Stable along one direction, unstable along the other: the upright pendulum.
- Centre: purely imaginary, closed orbits, never settles: the frictionless pendulum near the bottom.

These are four **canonical** phase portraits, the ones that matter most in control. They are not the whole list: there are also unstable nodes and foci, repeated-eigenvalue and degenerate nodes, and whole lines of equilibria when $\boldsymbol{A}$ is singular. Session 1 showed the pendulum doing several of these at once in different regions of the same plane. That is the difference.

Slotine & Li, 1991, §2.2; Khalil, 2002, §2.1.

Drawing a phase portrait yourself

```python
import numpy as np
import matplotlib.pyplot as plt

A = np.array([[-1.,  1.],      # <-- change this
              [-4., -1.]])

g = np.linspace(-2, 2, 300)
X1, X2 = np.meshgrid(g, g)

# the vector field: xdot = A x, at every point
DX1 = A[0,0]*X1 + A[0,1]*X2
DX2 = A[1,0]*X1 + A[1,1]*X2

plt.streamplot(X1, X2, DX1, DX2, density=0.8)
plt.plot(0, 0, 'o')
plt.xlabel(r'$x_1$'); plt.ylabel(r'$x_2$')
print(np.linalg.eigvals(A))
plt.show()
```

Code 2.3

The plot is nothing but the vector field of $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$ drawn at every point of the plane, with `streamplot` joining the arrows into curves.

Try these four and match each to a picture from the previous slide:

$\begin{bmatrix} -1 & 1 \\ -4 & -1 \end{bmatrix}$ focus

$\begin{bmatrix} -1 & 0 \\ 0 & -3 \end{bmatrix}$ node

$\begin{bmatrix} 1 & 0 \\ 0 & -2 \end{bmatrix}$ saddle

$\begin{bmatrix} 0 & 2 \\ -2 & 0 \end{bmatrix}$ centre

Print the eigenvalues each time and check they agree with what you see.

Putting it together on the example

The mass-spring-damper $m\ddot{q} + c\dot{q} + kq = u$ with $m = 1$, $c = 3$, $k = 2$:

$$\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}, \qquad \lambda = -1,\ -2$$

- Both real and negative: stable node, no oscillation.
- Slow mode $\lambda = -1$, time constant 1 s, settling in roughly 4 s.
- $\zeta = c/(2\sqrt{mk}) = 1.06$: only *slightly* overdamped. From $q(0) = 1$, $\dot{q}(0) = 0$ the position returns monotonically.

Change one number

Take $c = 0.4$ instead of 3. Then $\lambda^2 + 0.4\lambda + 2 = 0$ gives $\lambda = -0.2 \pm j1.40$.

The pair is now complex, so the mass oscillates, with $\omega_n = 1.41\,\text{rad/s}$ and $\zeta = 0.14$. Still stable, because $\sigma = -0.2 < 0$, but it rings for a long time before settling.

One coefficient moved the system from node to focus.

3. State feedback

Adding the input, step by step

We want to solve $\boxed{\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{u}}$ with $\boldsymbol{u}(t)$ given. The trick is the integrating factor used for scalar first-order ODEs, with $e^{-\boldsymbol{A}t}$ in the role of e^{-at}.

1. Move the state term to the left:

$$\dot{\boldsymbol{x}} - \boldsymbol{A}\boldsymbol{x} = \boldsymbol{B}\boldsymbol{u}$$

2. Multiply on the left by $e^{-\boldsymbol{A}t}$:

$$e^{-\boldsymbol{A}t}\dot{\boldsymbol{x}} - e^{-\boldsymbol{A}t}\boldsymbol{A}\boldsymbol{x} = e^{-\boldsymbol{A}t}\boldsymbol{B}\boldsymbol{u}$$

3. Recognise the left side as a single derivative. Because $\boldsymbol{A}$ commutes with $e^{-\boldsymbol{A}t}$,

$$\frac{d}{dt}\left(e^{-\boldsymbol{A}t}\boldsymbol{x}\right) = e^{-\boldsymbol{A}t}\dot{\boldsymbol{x}} - \boldsymbol{A}e^{-\boldsymbol{A}t}\boldsymbol{x}$$

which is exactly the left side. So

$$\frac{d}{dt}\left(e^{-\boldsymbol{A}t}\boldsymbol{x}(t)\right) = e^{-\boldsymbol{A}t}\boldsymbol{B}\boldsymbol{u}(t)$$

4. Integrate both sides from 0 to t. The left side telescopes:

$$e^{-\boldsymbol{A}t}\boldsymbol{x}(t) - \boldsymbol{x}(0) = \int_0^t e^{-\boldsymbol{A}s}\boldsymbol{B}\boldsymbol{u}(s)\,ds$$

5. Multiply on the left by $e^{\boldsymbol{A}t}$ and push it inside the integral, which is legal because it does not depend on $\boldsymbol{s}$:

$$\boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{x}(0) + \int_0^t e^{\boldsymbol{A}t}e^{-\boldsymbol{A}s}\boldsymbol{B}\boldsymbol{u}(s)\,ds$$

6. Finally $e^{\boldsymbol{A}t}e^{-\boldsymbol{A}s} = e^{\boldsymbol{A}(t-s)}$, since $\boldsymbol{A}t$ and $-\boldsymbol{A}s$ commute:

$$\boxed{\boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{x}(0) + \int_0^t e^{\boldsymbol{A}(t-s)}\boldsymbol{B}\boldsymbol{u}(s)\,ds}$$

Where the name comes from: unforced, the solution is $\boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{c}$ with $\boldsymbol{c} = \boldsymbol{x}(0)$ a *constant*. The method keeps that shape but lets the constant vary, $\boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{c}(t)$, and solves for $\boldsymbol{c}(t)$. Steps 3 and 4 are exactly that: $\boldsymbol{c}(t) = e^{-\boldsymbol{A}t}\boldsymbol{x}(t)$ is the varying constant, and we found $\dot{\boldsymbol{c}} = e^{-\boldsymbol{A}t}\boldsymbol{B}\boldsymbol{u}$. Hence **variation of constants**, or variation of parameters.

Reading the forced response

$$\boldsymbol{x}(t) \;=\; \underbrace{e^{\boldsymbol{A}t}\boldsymbol{x}(0)}_{\text{free response}} \;+\; \underbrace{\int_0^t e^{\boldsymbol{A}(t-s)}\boldsymbol{B}\boldsymbol{u}(s)\,ds}_{\text{forced response}}$$

The integral is bookkeeping. Read the integrand from the inside out, for one past instant s:

- $\boldsymbol{u}(s)$: what we commanded at s, and $\boldsymbol{B}\boldsymbol{u}(s)$: how much of it entered the state;
- $e^{\boldsymbol{A}(t-s)}$: that contribution propagated for the remaining time $t-s$, then $\int_0^t \cdots ds$ adds up every past instant.

The state at t is the initial condition propagated forward plus the accumulated echo of everything the input has done since.

Definition: Hurwitz

$\boldsymbol{A}$ is **Hurwitz** when every eigenvalue has strictly negative real part, $\operatorname{Re}\lambda_i < 0$ for all i. Used constantly from here on.

If $\boldsymbol{A}$ is Hurwitz the free response converges to zero for *every* $\boldsymbol{x}(0)$, so the asymptotic behaviour stops depending on it. If not, there *exist* initial conditions for which it does not vanish, though a particular $\boldsymbol{x}(0)$ may still decay if it misses the bad modes. Note this removes the dependence on $\boldsymbol{x}(0)$, not the forced term: for arbitrary $\boldsymbol{u}$ the forced response need not settle either.

If we do not like the eigenvalues, we change them

So far $\boldsymbol{A}$ was given and we read off its eigenvalues. Now we choose the input. Take $\boldsymbol{u} = -\boldsymbol{K}\boldsymbol{x}$ with $\boldsymbol{K} \in \mathbb{R}^{m \times n}$, and substitute into $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{u}$:

The closed loop

$$\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}(-\boldsymbol{K}\boldsymbol{x}) = (\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K})\,\boldsymbol{x}$$

- The closed loop is again linear, with matrix $\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K}$.
- So its stability is again decided by eigenvalues, now those of $\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K}$.
- Design question: if $(\boldsymbol{A}, \boldsymbol{B})$ is controllable, choose $\boldsymbol{K}$ so that the eigenvalues of $\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K}$ sit at desired locations. We make that condition precise in two slides.

An assumption we are making silently

State feedback assumes the complete state $\boldsymbol{x}$ is available. That is rarely true: on the pendulum we may measure θ but not $\dot{\theta}$. Module 3 reconstructs what we cannot measure.

Designing the gain, step by step

Take the double integrator, a mass pushed by a force, with no spring and no friction:

$$\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad \boldsymbol{B} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Open loop $\lambda = 0, 0$: it drifts and never returns. Let $\boldsymbol{K} = [\, k_1 \;\; k_2 \,]$.

1. Closed loop: $\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K} = \begin{bmatrix} 0 & 1 \\ -k_1 & -k_2 \end{bmatrix}$.

2. Its characteristic polynomial, from $\det(\lambda \boldsymbol{I} - (\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K})) = 0$:

$$\det \begin{bmatrix} \lambda & -1 \\ k_1 & \lambda + k_2 \end{bmatrix} = \lambda^2 + k_2 \lambda + k_1$$

3. Match the target $\lambda^2 + 2\zeta\omega_n\lambda + \omega_n^2$ coefficient by coefficient: $k_1 = \omega_n^2$, $k_2 = 2\zeta\omega_n$.

Numbers

Ask for $\omega_n = 3\,\text{rad/s}$ and $\zeta = 0.7$:

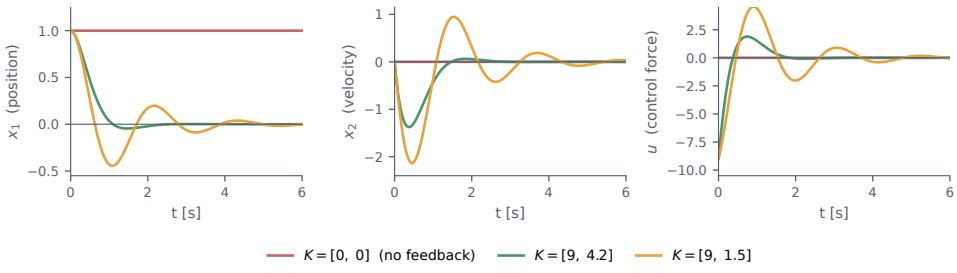
$$k_1 = 9, \qquad k_2 = 4.2$$

which places the closed-loop eigenvalues at $\lambda = -2.10 \pm j2.14$.

The gains are not tuned by trial and error. They are read off by matching two polynomials.

What the gain buys, and what it costs

Double integrator $\boldsymbol{A} = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$, $\boldsymbol{B} = \left[\begin{smallmatrix} 0 \\ 1 \end{smallmatrix}\right]$ (x_1 position, x_2 velocity), from $\boldsymbol{x}(0) = \left[\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right]$, with $\boldsymbol{u} = -\boldsymbol{K}\boldsymbol{x} = -k_1 x_1 - k_2 x_2$, closed loop $\dot{\boldsymbol{x}} = (\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K})\boldsymbol{x}$:



$\boldsymbol{K} = [0, 0]$: $\lambda = 0, 0$, never returns. $\boldsymbol{K} = [9, 4.2]$: $\zeta = 0.7$. $\boldsymbol{K} = [9, 1.5]$: $\zeta = 0.25$, rings. With $\boldsymbol{K} = \boldsymbol{0}$ both states stay put only because $x_2(0) = 0$; in general the uncontrolled double integrator drifts, $x_1(t) = x_1(0) + x_2(0)t$. Driving x_1 to zero costs a velocity excursion, and $[9, 1.5]$ pays the larger one, -2.1 against -1.4. Careful with the reason: its poles are *slower* ($\mathrm{Re} = -0.75$ against -2.1), so the cause is the lower damping ratio, not extra speed. Third panel: both gains start at $\boldsymbol{u}(0) = -k_1 x_1(0) = -9$, but the badly damped one swings back to $+4.6$ against $+1.9$, fighting itself. Real actuators saturate, which is the practical limit on $\boldsymbol{K}$.

When can the eigenvalues be placed at all?

Controllability test

$(\boldsymbol{A}, \boldsymbol{B})$ is **controllable** if and only if $\mathcal{C} = \begin{bmatrix} \boldsymbol{B} & \boldsymbol{AB} & \cdots & \boldsymbol{A}^{n-1}\boldsymbol{B} \end{bmatrix}$ has rank n.

If it does, real closed-loop poles may be placed arbitrarily, while complex ones must be assigned in conjugate pairs, because $\boldsymbol{A}$, $\boldsymbol{B}$ and $\boldsymbol{K}$ are real. If it does not, some directions of the state cannot be reached through $\boldsymbol{B}$, and no gain moves their eigenvalues.

Controllable

Double integrator, $\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$, $\boldsymbol{B} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$: $\mathcal{C} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, rank 2. Yes

Force changes velocity, velocity changes position: all reachable.

Not controllable

Two decoupled stable first-order modes, first one actuated, $\boldsymbol{A} = \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix}$, $\boldsymbol{B} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$: $\mathcal{C} = \begin{bmatrix} 1 & -1 \\ 0 & 0 \end{bmatrix}$, rank 1. No

Second row all zeros: x_2 never hears the input.

```python
def ctrb(A, B):
    n = A.shape[0]
    return np.hstack(
      [np.linalg.matrix_power(A,k) @ B
       for k in range(n)])

print(np.linalg.matrix_rank(ctrb(A,B)))
```

Code 2.4

The unreachable mode above is $\lambda = -2$, already stable, so the system is still usable. That weaker property is called *stabilizability*.

Åström & Murray, 2008, §7.1.

4. Simulation lab

Lab 2: the linear toolbox

Everything today is three library calls. Build the toolbox now; you will reuse it in every remaining session.

`np.linalg.eigvals`	eigenvalues
`scipy.linalg.expm`	$e^{\boldsymbol{A}t} = \boldsymbol{\Phi}(t)$
`solve_ivp`	simulation

The same $\boldsymbol{A}$ appears here and in the worked example, so you can check your code against the closed form we derived in class.

Code 2.5

```
import numpy as np
from scipy.linalg import expm
from scipy.integrate import solve_ivp

A = np.array([[ 0., 1.],
              [-2., -3.]])
B = np.array([[0.], [1.]])

print(np.linalg.eigvals(A))   # -1, -2
print(expm(A*0.5))            # Phi(0.5)

sol = solve_ivp(lambda t, x: A @ x,
                [0, 6], [1., 0.],
                max_step=0.01)
```

Lab 2: what to do

1. Simulate $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$ from $\boldsymbol{x}(0) = [1, 0]^\top$, $[0, 1]^\top$ and $[1, -2]^\top$. Do they settle at the same rate?
2. Change the *damping coefficient* c from 3 to 0.4, recompute the eigenvalues and simulate. Read ω_d, ω_n, ζ and compare with the formulas.
3. Double integrator (redefine $\boldsymbol{A}$ and $\boldsymbol{B}$ as in Code 2.6): implement $\boldsymbol{u} = -\boldsymbol{K}\boldsymbol{x}$ with $\boldsymbol{K} = [9, 4.2]$ and verify the closed-loop eigenvalues.
4. Design $\boldsymbol{K}$ for $\omega_n = 6$ rad/s, $\zeta = 0.7$, and simulate. Plot $\boldsymbol{u}(t)$ and report its peak.

This lab is Homework 2.

Code 2.6: Point 3 of the Lab 2

```
# double integrator, NOT the A above
A = np.array([[0., 1.],
              [0., 0.]])
B = np.array([[0.], [1.]])

K   = np.array([[9.0, 4.2]])
Acl = A - B @ K
print(np.linalg.eigvals(Acl))
sol = solve_ivp(lambda t, x: Acl @ x,
        [0, 6], [1., 0.], max_step=0.01)
u = -(K @ sol.y).ravel()
```

The point of item 5

With $\boldsymbol{K} = [\,\omega_n^2 \;\; 2\zeta\omega_n\,]$ and $\boldsymbol{x}(0) = [1,\ 0]^\top$, $\boldsymbol{u}(0) = -\boldsymbol{K}\boldsymbol{x}(0) = -\omega_n^2(1) - 2\zeta\omega_n(0) = -\omega_n^2$: the k_2 term drops out *only* because $x_2(0) = 0$. So with the same ζ and $\boldsymbol{x}(0)$, doubling ω_n makes the response about twice as fast and the initial effort exactly four times larger, -9 to -36. Does $|\boldsymbol{u}|$ peak at $t = 0$ in your plot?

A bridge to Module 2: from eigenvalues to Lyapunov functions

The eigenvalue test exists only for linear systems. There is a second route to the same conclusion, and *that* one survives into the nonlinear world.

The Lyapunov equation

$$\boldsymbol{A} \text{ Hurwitz} \iff \text{for every } \boldsymbol{Q} = \boldsymbol{Q}^\top > 0 \text{ there is a unique } \boldsymbol{P} = \boldsymbol{P}^\top > 0 \text{ with } \boldsymbol{A}^\top \boldsymbol{P} + \boldsymbol{P}\boldsymbol{A} = -\boldsymbol{Q}$$

Take $V(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{P}\boldsymbol{x}$, positive for $\boldsymbol{x} \neq \boldsymbol{0}$. Along solutions,

$$\dot{V}(\boldsymbol{x}) = \boldsymbol{x}^\top \left(\boldsymbol{A}^\top \boldsymbol{P} + \boldsymbol{P}\boldsymbol{A}\right)\boldsymbol{x} = -\boldsymbol{x}^\top \boldsymbol{Q}\boldsymbol{x} < 0$$

So V decreases along every trajectory: stability without computing an eigenvalue.

On our example

$\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$ with $\boldsymbol{Q} = \boldsymbol{I}$ gives $\boldsymbol{P} = \begin{bmatrix} 1.25 & 0.25 \\ 0.25 & 0.25 \end{bmatrix}$, whose eigenvalues 0.19 and 1.31 are both positive.

In Python: `solve_continuous_lyapunov(A.T, -Q)`.

Why this matters: eigenvalues, exponential stability and quadratic Lyapunov functions are three views of one fact. For nonlinear systems exponential stability still makes sense and Jacobian eigenvalues still give *local* information near hyperbolic equilibria; what is lost is the global matrix-exponential solution and the superposition of modes. Lyapunov functions keep working and reach nonlocal conclusions.

Khalil, 2002, §4.1; Slotine & Li, 1991, §3.5.

Wrap-up and next session

Today

- $\boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{x}(0)$, with $e^{\boldsymbol{A}t}$ the state transition matrix;
- for diagonalizable $\boldsymbol{A}$, the solution as a sum of modes $e^{\lambda_i t}\boldsymbol{v}_i$, with generalized modes $t^k e^{\lambda t}$ in the defective case;
- exponential stability from $\operatorname{Re}\lambda_i < 0$, globally and cheaply;
- σ decays, ω_d oscillates, and the pair (ω_n, ζ);
- $\boldsymbol{u} = -\boldsymbol{K}\boldsymbol{x}$ gives $\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K}$, whose eigenvalues we place.

Next session: back to nonlinear

- equilibrium points of $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$;
- the pendulum has two, and they are not alike;
- local linearization: building an $\boldsymbol{A}$ that is valid *near* an equilibrium;
- what that $\boldsymbol{A}$ tells us, and where it stops telling the truth.

Everything from today gets reused there. The linearized pendulum will be a 2×2 matrix whose eigenvalues you already know how to read.

Homework 2, due next session

Complete Items 1–4 of Lab 2. Submit a **single PDF** containing your plots, relevant code, and brief explanations.

A submission link or section will be available under *Homework 2* on Blackboard. Upload your work there by the posted deadline. **Submissions by email will not be accepted.**